

Problem 1 - Logistic Regression for Diabetes Dataset

Using `train_test_split` method from `sklearn.model_selection` the diabetes data is split into a training set 80% and 20% validation set. Standardization is immediately applied afterwards using the `sklearn.preprocessing` library.

Assumptions made: $\alpha = 0.01$, $\lambda = 0$, and iterations = 10000. It is observed that more iterations were needed than the previous 2000 iterations used for the housing cost prediction. Random state = 4 for both datasets.

Next, the `compute_cost` method is updated with the convex logistic regression cost function and uses the newly added sigmoid method. The new sigmoid and updated `compute_cost` method can be seen in Figures 1.1 & 1.2 respectively. The output of the `compute_cost` method is used to create a plot of the cost history for the training and validation datasets. This cost history plot can be seen in Figure 1.6. Note the histories for both training and validation sets decrease in tandem showing no signs of over fitting with a convergence at roughly 4000 iterations.

```
def sigmoid(z):  
    return 1 / (1 + np.exp(-z))
```

Figure 1.1 - New sigmoid function

```
def compute_cost(X, Y, theta):  
    h = sigmoid(X.dot(theta))  
    return -np.mean(Y * np.log(h) + (1 - Y) * np.log(1 - h))
```

Figure 1.2 - Updated `compute_cost` method

An accuracy method is also added to make decisions by using the sigmoid method with a 0.5 threshold for the prediction of the binary classification. The `gradient_descent` method is updated to support the new sigmoid and `compute_accuracy` methods and can be seen in Figures 1.3 & 1.4 respectively. The `compute_accuracy` method is solely used for plotting the accuracy history for both the training and validation datasets. A separate variable `Y_pred` is created and used to store a list of predicted classifications using the validation dataset. This list of predicted values can be seen in Figure 1.5 and is used in code explained later. Notice it also uses the same threshold used in the `compute_accuracy` method.

```
def compute_accuracy(X, Y, theta):
    predictions = (sigmoid(X.dot(theta)) >= 0.5).astype(int)
    return np.mean(predictions == Y)
```

Figure 1.3 - New compute_accuracy method

```
def gradient_descent(X_training, Y_training, X_validation, Y_validation, theta,
                    alpha, lamda, iterations):
    m = len(Y_training)
    cost_history_training = np.zeros(iterations)
    cost_history_validation = np.zeros(iterations)
    acc_history_training = np.zeros(iterations)
    acc_history_validation = np.zeros(iterations)

    for i in range(iterations):
        predictions = sigmoid(X_training.dot(theta))
        errors = np.subtract(predictions, Y_training)

        theta_copy = np.copy(theta)
        theta_copy[0] = 0

        sum_delta = (alpha / m) * (X_training.transpose().dot(errors) + (lamda * theta_copy))
        theta = theta - sum_delta;

        cost_history_training[i] = compute_cost(X_training, Y_training, theta)
        cost_history_validation[i] = compute_cost(X_validation, Y_validation, theta)
        acc_history_training[i] = compute_accuracy(X_training, Y_training, theta)
        acc_history_validation[i] = compute_accuracy(X_validation, Y_validation, theta)

    return (theta, cost_history_training, cost_history_validation,
            acc_history_training, acc_history_validation)
```

Figure 1.4 - Updated gradient_descent method

```
Y_pred = (sigmoid(X_test.dot(theta)) >= 0.5).astype(int)
```

Figure 1.5 - List of class predictions

The sklearn metrics library is used for calculating and displaying the accuracy, precision, recall, and F1 values, all of which take Y_test (the validation outcomes) and Y_pred (the predicted outcomes discussed earlier) as parameters. The final values of each along with the final theta values are:

F1 Score 0.7

Recall 0.6730769230769231

Precision 0.7291666666666666

Final test accuracy = 0.8051948051948052

Final training accuracy = 0.7736156351791531

Final theta = [-0.85673274 0.46339266 0.97556647 -0.30344819 0.0096602 -0.04405247
0.75164618 0.39689997 0.22910548]

The F1 score can be found by dividing the doubled product of the precision and recall by the sum of the precision and recall.

Recall can be found by dividing the true positive outcomes by the total number of positive outcomes (true positives + false negatives).

Precision can be found by swapping false negatives in the recall formula with false positives.

Finally, accuracy is simply the quotient of the correct predictions and total predictions. The accuracy history of the diabetes dataset can be seen in Figure 1.7. Note that both the training and validation accuracies increase in tandem showing no signs of overfitting. These same calculations are used later in the regularized logistic regression model used for the cancer dataset.

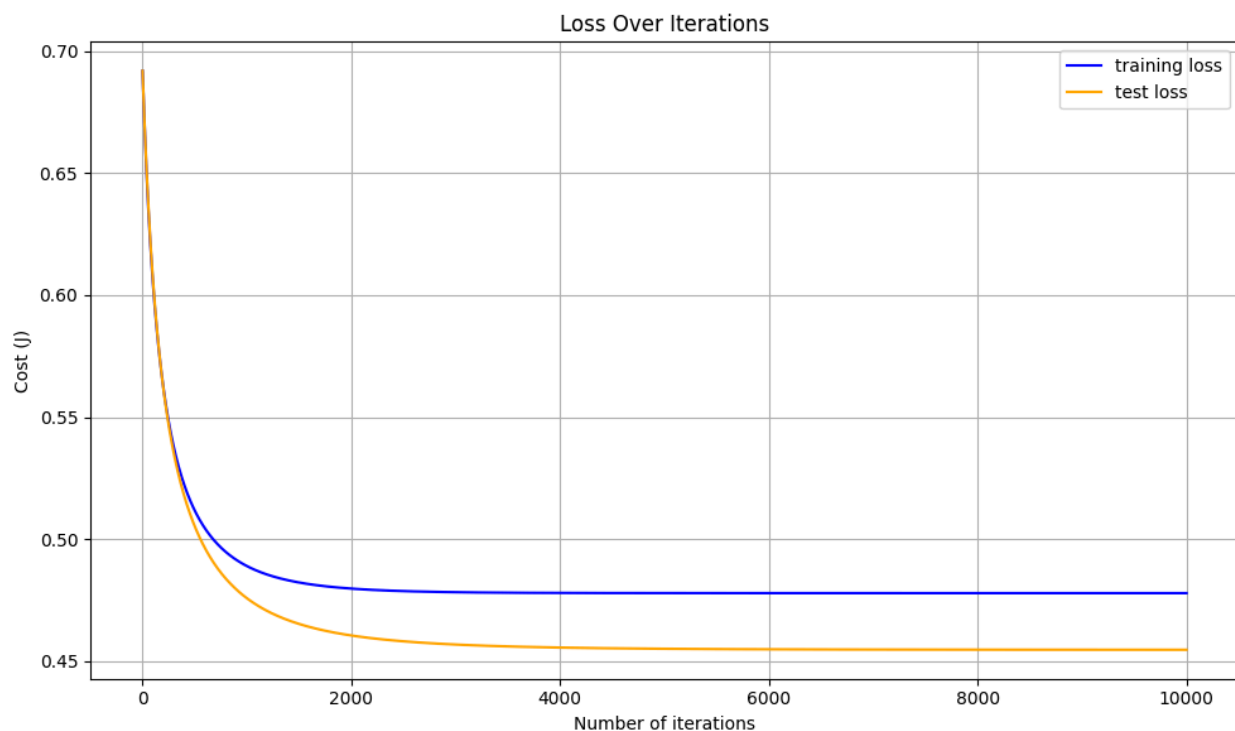


Figure 1.6 - Diabetes dataset cost history

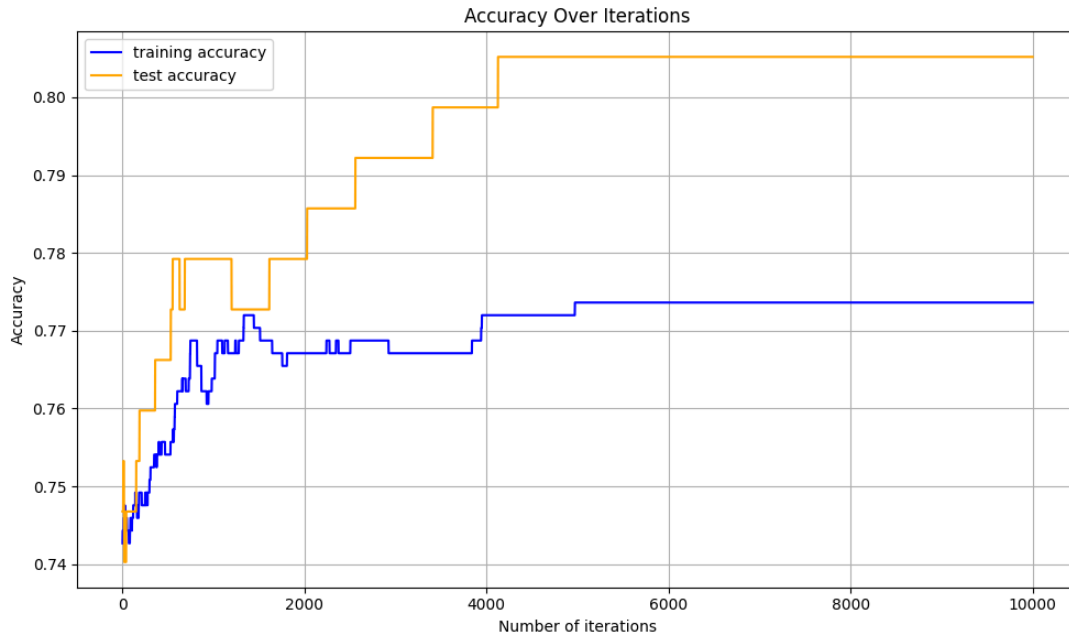


Figure 1.7 - Diabetes dataset accuracy history

The same sklearn library is also used to create a confusion matrix and likewise takes Y_{test} and Y_{pred} as parameters. The confusion matrix is displayed using the seaborn library along with a heatmap seen in Figure 1.8. There are more false positives and false negatives in the diabetes confusion matrix than in the confusion matrix obtained from the cancer dataset shown in Figure 2.6 (using the regularized logistic regression model). This may be attributed to the fewer explanatory variables in the diabetes dataset, as the cancer dataset has over triple the independent variables.

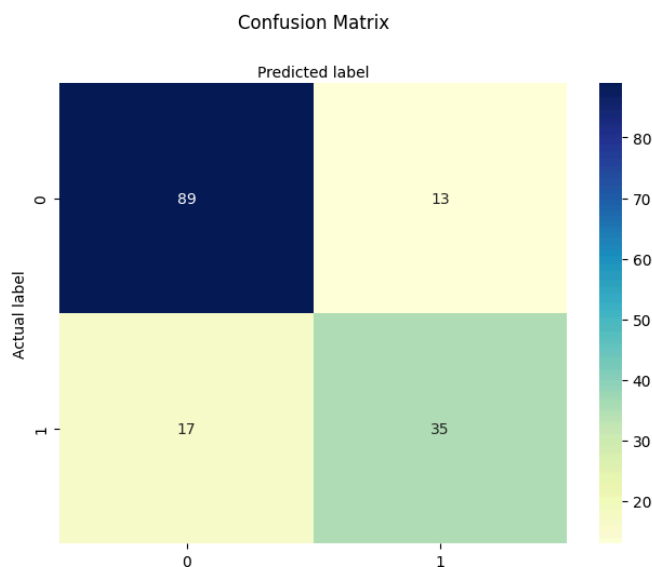


Figure 1.8 - Diabetes confusion matrix with heatmap

Problem 2 - Regularized Logistic Regression for Cancer Dataset

The same design choices made in problem one were made in problem 2. The same sigmoid function shown in Figure 1.1 is used. The same updated compute_cost method is used and can be seen in Figure 1.2. Even the same gradient_descent and Y_pred list are used. The cost history and accuracy history for both non-regularized and regularized models can be found in Figure 2.1, 2.2, 2.3, and 2.4 respectively.

The main difference is that the model used for the cancer dataset in part 2 is regularized, that means lambda (the cost penalty scaler) is increased from 0 to 10. This noticeably improved the performance of the model. The regularization made much more of a difference on the cancer dataset than on the housing dataset from homework 2. This may be attributed to the fact that more explanatory variables increase lambda's impact on the cost. As seen below, the F1 score, recall, precision, and test accuracy were all higher with a lambda value of 10 with training accuracy being the exception as it is the same for both models.

F1 Score 0.9444444444444444

Recall 1.0

Precision 0.8947368421052632

Final test accuracy = 0.9649122807017544

Final training accuracy = 0.9868131868131869

Final theta = [-0.07093701 0.59452508 0.59857595 0.57726223 0.60277665 0.19960553
-0.11776321 0.61462863 0.66100463 0.08635774 -0.39905754 0.89564666
-0.00812421 0.67560219 0.67074971 0.07376374 -0.46861889 -0.02353271
0.13117013 -0.3174654 -0.54997294 0.90049513 0.93036484 0.81944273
0.82944951 0.68802059 0.13825449 0.6966356 0.7436808 0.65356729
0.17142196]

lambda = 10 f1 Score 0.9855072463768116

lambda = 10 recall 1.0

lambda = 10 precision 0.9714285714285714

Final lambda = 10 test accuracy = 0.9912280701754386

Final lambda = 10 training accuracy = 0.9868131868131869

Final lambda = 10 theta = [-0.26434652 0.38943204 0.40329885 0.3792292 0.37972159
0.12410189
0.01051426 0.35853736 0.40003012 0.08022491 -0.2237938 0.47741648
-0.02478888 0.3677437 0.36286732 0.02116957 -0.20219909 -0.00400752
0.09873596 -0.16648743 -0.25962945 0.53280497 0.54025672 0.49243706
0.48001767 0.37133198 0.14217659 0.40252688 0.45722521 0.38176075
0.12530328]

The increase in model accuracy is not only observed by the values mentioned earlier, but can also be seen in the cost history plots shown in Figures 2.1 and 2.3. With the regularized model we can see a clear convergence but when looking at the non-regularized model it seems more iterations are needed for the cost to completely level off.

When looking at the accuracy history plots shown in Figures 2.2 and 2.4 respectively, it can be observed that the training and validation histories for the regularized model more closely resemble each other. But for the non-regularized history plot, the training and validation accuracy are further from each other and they show signs of mild overfitting but that is fixed with regularization.

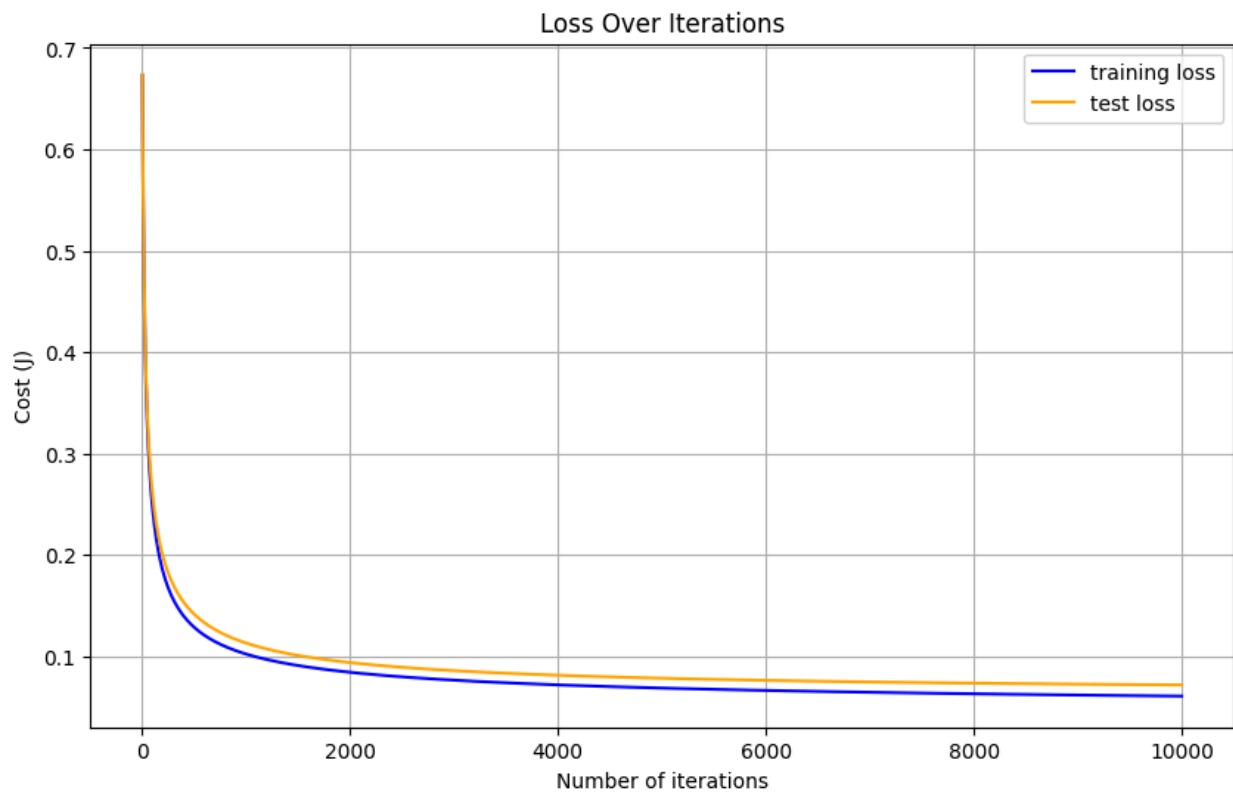


Figure 2.1 - Cancer dataset cost history (not regularized)

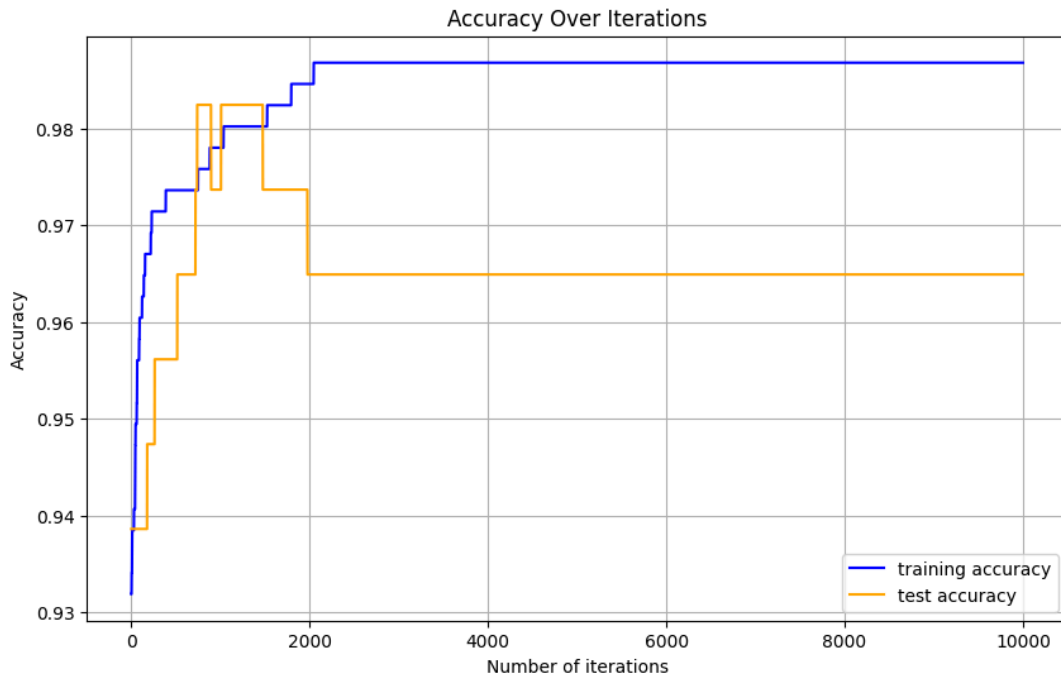


Figure 2.2 - Cancer dataset accuracy history (not regularized)

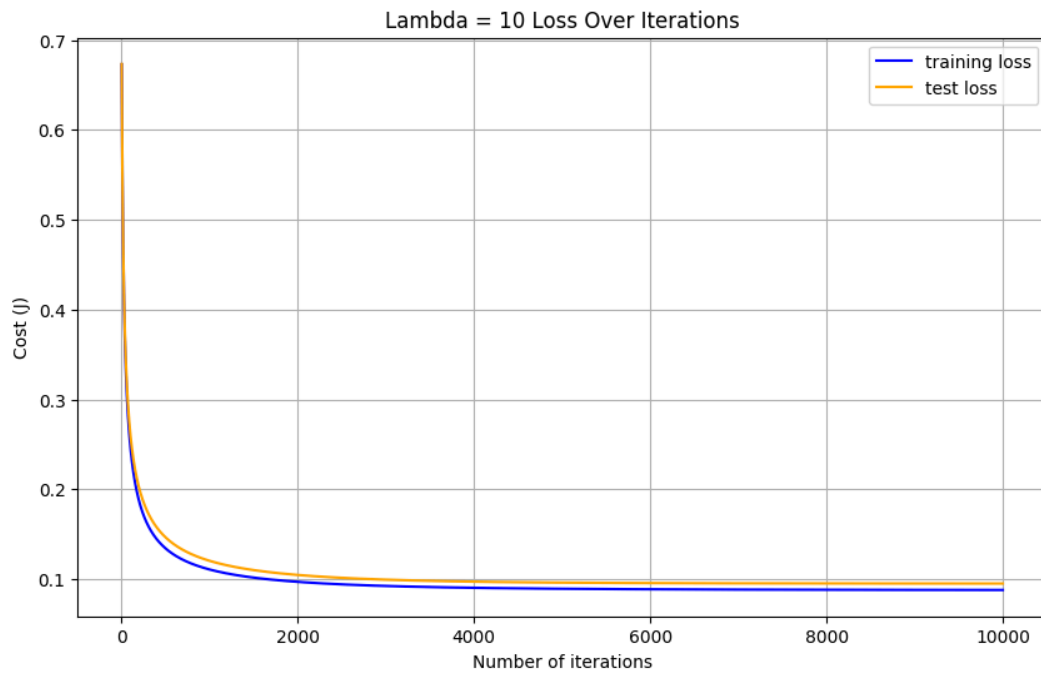


Figure 2.3 - Cancer dataset cost history (regularized)

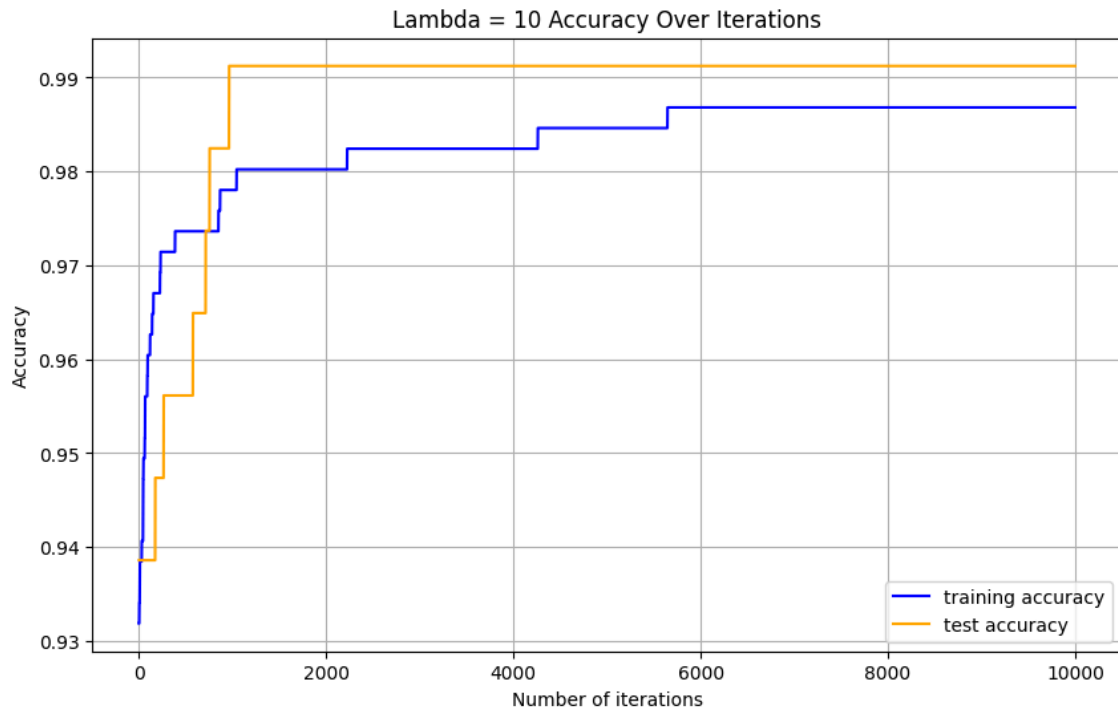


Figure 2.4 - Cancer dataset accuracy history (regularized)

As one final metric for observing the performance of the binary classification, specifically the differences between the regularized and non regularized models, we can clearly see that the false positives decrease from 4 to 1. This matches with the changes in F1 score, precision, accuracy, etc.

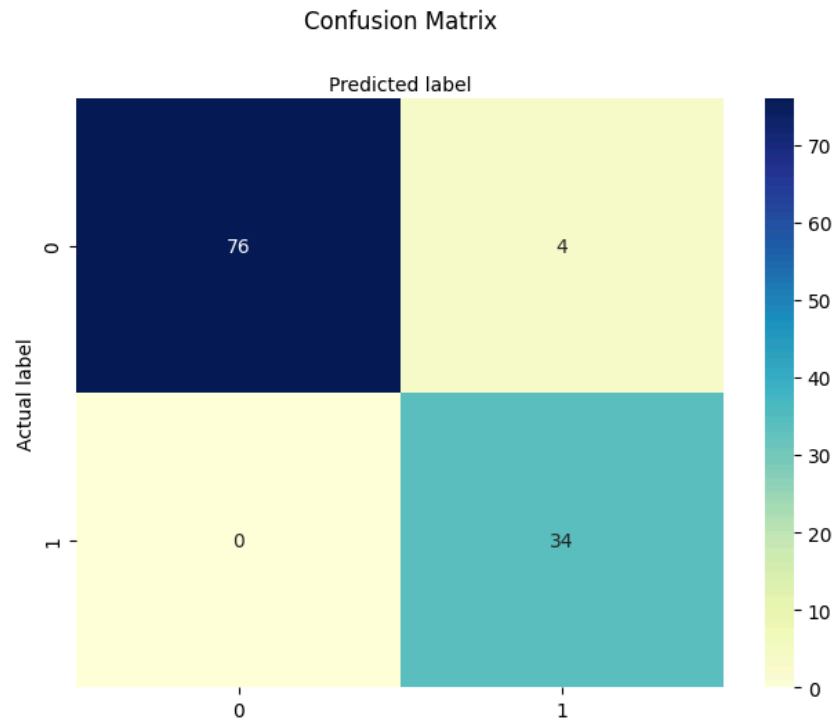


Figure 2.5 - Cancer dataset confusion matrix (not regularized)

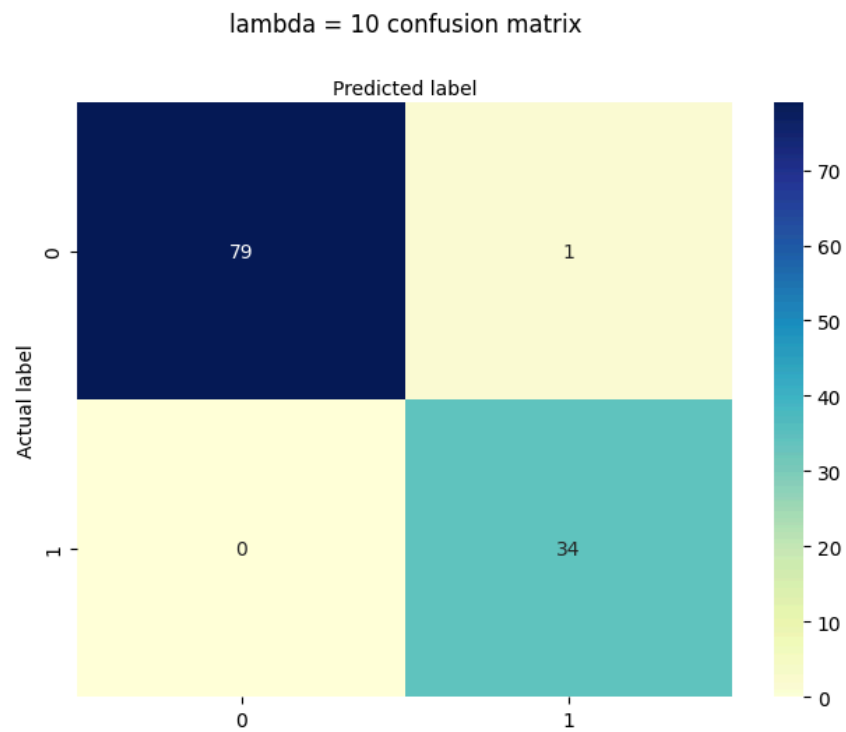


Figure 2.6 - Cancer dataset confusion matrix (regularized)